Skills

Nowe zadanie?
Najpierw instrukcja!



Tool vs Skill

Tool

Co mogę zrobić?

- find_building
- get_geometry
- confirm_entrance

korzysta z

Skill

Jak mam to zrobić?

1. Znajdź budynek
2. Pobierz geometrię
3. Zapytaj o wejście
4. Zaproponuj punkt
5. Potwierdź i zapisz

Anatomia Skilla

tup-zak/

- SKILL.md
- scripts/ → kod do uruchomienia
- references/ → szczegóły na żądanie
- assets/ → szablony, schematy

SKILL.md

```
---
name: tup-zak
description: Oznaczanie wejść do
 budynków. Użyj, gdy...
---
# Kroki
1. ...
2. ...
# Zasady
```

Co robi + kiedy go użyć

Progressive disclosure

1. Metadane — name + description

zawsze
~100 tokenów / Skill

↓

2. Instrukcje — SKILL.md

gdy Skill pasuje
< 5000 tokenów

↓

3. Zasoby — scripts/, references/, assets/

gdy instrukcja tego wymaga

Spis treści zawsze, rozdział na żądanie.

Skills a MCP

	Tools	Resources	Prompts	Skills
Pytanie	Co mogę zrobić?	Jakie mam dane?	Jak to opisać?	Jak to zrobić?
Kto wybiera?	model	aplikacja	użytkownik	model
Kiedy w kontekście?	definicje zawsze	gdy aplikacja dołączy	gdy użytkownik wybierze	opis zawsze, treść na żądanie

MCP daje agentowi ręce. Skill daje przepis.

Skill w akcji: wejście do MiNI

Skill: tup-zak „Pomóż mi oznaczyć wejście do MiNI”

1. find_building → 2. get_geometry → 💬 3. Które wejście?

↓

4. propose_entrance + walidacja → 💬 5. Zapisać? → 6. confirm_entrance

▢ MCP Tool ▢ dialog z użytkownikiem

Skill wyznacza kolejność. Narzędzia wykonują kroki.

Code Execution

Gdy agent nie ma gotowego narzędzia.



Następny wykład:

Code Execution — od przepisu do własnego narzędzia → **Tools** — Co mogę zrobić? + **Skills** — Jak to zrobić? → **Code Execution** — Nie ma narzędzia? Napiszę je!